Predictive Analytics for Equity Markets

Sayantika Nag Class Project for CS252, Winter 2026

Introduction and Goal of the Project:

Financial markets generate large volumes of time dependent data characterized by volatility, noise, and complex temporal dependencies. Predicting stock prices remains one of the most challenging problems in quantitative finance due to market efficiency, stochastic behavior, and external macroeconomic influences. Accurate prediction of stock prices has significant importance in quantitative finance, algorithmic trading, and investment decision-making. Traditional statistical models assume linear relationships, while modern machine learning and deep learning models attempt to capture nonlinear patterns.

The goal of this project is to investigate the statistical properties and predictability of technology stock price data through a combination of exploratory financial analysis and predictive modeling. Before attempting to forecast future prices, it is essential to understand the underlying structure, trends, and risk characteristics of the historical data. Therefore, this project is organized into two complementary components.

Exploratory Financial Analysis:

The first phase of the project focuses on understanding the historical behavior of selected technology stocks. This stage aims to characterize trend patterns, return distributions, and risk exposure using established financial metrics.

H1: Long-Term Trend Behavior: It is hypothesized that major technology stocks demonstrate sustained upward long-term trends driven by sector growth and innovation, while also displaying significant short-term volatility. Time-series visualizations and trend inspection will be used to assess directional movement and price evolution over the observation period.

H2: Daily Return Characteristics: Daily returns are expected to fluctuate around a small mean value with substantial variability. Consistent with financial theory, returns are hypothesized to exhibit near-zero mean behavior but non-constant variance. Summary statistics and distributional analysis will evaluate these characteristics.

H3: Moving Average Trend Smoothing: We hypothesize that moving averages (e.g., 10-day, 20-day, 50-day) will reduce short-term noise and highlight clearer directional trends. Comparing short-term and long-term moving averages may also reveal crossover signals indicating momentum shifts.

H4: Downside Risk and Value at Risk (VaR): We hypothesize that Value at Risk (VaR) estimates will demonstrate measurable downside exposure, reflecting volatility clustering and non-normal return distributions typical of financial data. Risk metrics will quantify potential losses under normal market conditions.

Predictive Modeling Evaluation:

Following the exploratory financial analysis, the second phase of the study evaluates whether historical stock price data contains sufficient temporal structure to enable meaningful forecasting. At this phase we plan to compare models of increasing complexity to determine whether advanced methods provide measurable improvement over simpler approaches.
To ensure a rigorous and hierarchical comparison, models of increasing complexity are to be implemented.

We are planning to start with the simple Random Walk model as the primary naive baseline, assuming that tomorrow's closing price equals today's price. Next, a classical ARIMA model to capture potential linear dependencies in different price series. Moving beyond traditional statistical approaches, XGBoost regression is to be applied using engineered features such as lagged prices, moving averages, and rolling volatility to model nonlinear relationships. Finally, a Long Short-Term Memory (LSTM) neural network is to be implemented to learn sequential temporal dependencies directly from historical price data.

H5: LSTM model should do better than the simple models like Random Walk and ARIMA, and perform about the same as or better than XGBoost. However, because stock markets are very noisy and hard to predict, we do not expect the improvement to be very large.

Model performance is to be evaluated using time-series-consistent validation techniques and metrics such as RMSE and MAE.

Overall, the project aims to provide a comprehensive understanding of stock price behavior and to evaluate the effectiveness of modern machine learning and deep learning approaches in financial forecasting.

Prior Works on this topic:

Stock price forecasting has been widely studied in both econometrics and machine learning research. Traditional statistical models include the Random Walk model, which is based on the Efficient Market Hypothesis (EMH) and assumes that future price movements cannot be consistently predicted, ARIMA models that capture linear time-series relationships, and GARCH models that are commonly used to model volatility clustering in financial returns. While these approaches are grounded in strong theoretical foundations and remain important benchmarks, their ability to predict short-term stock prices is often limited due to nonlinear behavior, structural changes, and the high level of noise present in financial markets.

In recent years, machine learning and deep learning techniques have been increasingly applied to financial forecasting. Methods such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks are particularly suited for sequential data because they can capture temporal dependencies and nonlinear patterns in historical price movements. Empirical studies suggest that LSTM models may outperform traditional linear models under certain market conditions, especially when sufficient historical data and well-designed features are available. However, because financial markets are inherently stochastic and close to efficient, improvements in predictive performance are often moderate rather than dramatic.

Dataset:

The first step in this project is to obtain and load the required stock market data into memory. Historical stock price data is sourced from Yahoo Finance, a widely used platform that provides comprehensive financial market data and investment-related information. We used the *"yfinance"* Python library to do so. The *" yfinance"* package provides a convenient and efficient interface for downloading historical market data directly into Python in a structured format suitable for analysis. This allows seamless retrieval of daily stock prices, including open, high, low, close,and trading volume. The downloaded data is then stored as Pandas DataFrames for subsequent preprocessing, exploratory analysis, and modeling.

We are using the **Google stock** as the historical data for this project.
Key Characteristics:

- Time Period : **Last 5 years** (weekends and holidays were missing)
- Number of samples: approximately 1256 trading days
- Features: Date, Open price, High price, Low price, Close price,Volume
- **Close price** was used as the primary **target variable for prediction.**

Data Pre-processing steps:

- The raw data had multi-index thus I flattened it first and then handled the Date-Indexing.
- It does not have any missing values.
- Computed the log-returns.
- Train-test split using time-series order.
- Feature scaling for neural network models.

Summary Statistics:

Price	Close	High	Low	Open	Volume
count	1256.000000	1256.000000	1256.000000	1256.000000	1.256000e+03
mean	153.700494	155.402700	151.928883	153.606827	2.424869e+07
std	54.289613	54.914953	53.558544	54.291854	1.017008e+07
min	82.868477	85.905703	82.828774	84.873429	6.138200e+06
25%	118.592909	119.997502	116.823802	118.220826	1.764432e+07
50%	139.066978	140.414377	137.740424	138.978135	2.185355e+07
75%	171.259106	172.996237	169.057715	170.830107	2.800092e+07
max	344.899994	350.149994	338.589996	348.515015	9.779860e+07

The summary statistics provide an overview of the distribution and variability of the stock price over the observation period of 1,256 trading days. The average closing price is $153.70, with a standard deviation of $54.29, indicating substantial price variability and volatility over time. The minimum closing price of $82.87 and maximum of $344.90 show that the stock experienced significant long-term growth, supporting the hypothesis of an overall upward trend. The quartile values further confirm this progression, with the median

price at $139.07 and the 75th percentile at $171.26, indicating that most recent prices are higher than earlier observations. Additionally, the average trading volume of approximately 24.25 million shares, with large variation, suggests periods of increased market activity and investor interest. Overall, these statistics confirm the presence of both long-term growth and short-term volatility, which are characteristic features of technology stocks.

Experiments and Results:

A. Exploratory Analysis:

1.Historical View of Closing Prices:

The historical closing price plot shows that **Google's stock exhibits a clear long-term upward trend**, interspersed with periods of decline and recovery. The mean price level changes over time, indicating that the series is non-stationary. Additionally, the magnitude of price fluctuations increases in later periods, suggesting volatility clustering. **This supports hypothesis H1.**
Moreover, these characteristics confirm the presence of temporal structure and justify the use of time-series forecasting models.

2. Rolling Mean and Rolling Standard Deviation Analysis:

We are trying to answer the following questions here:

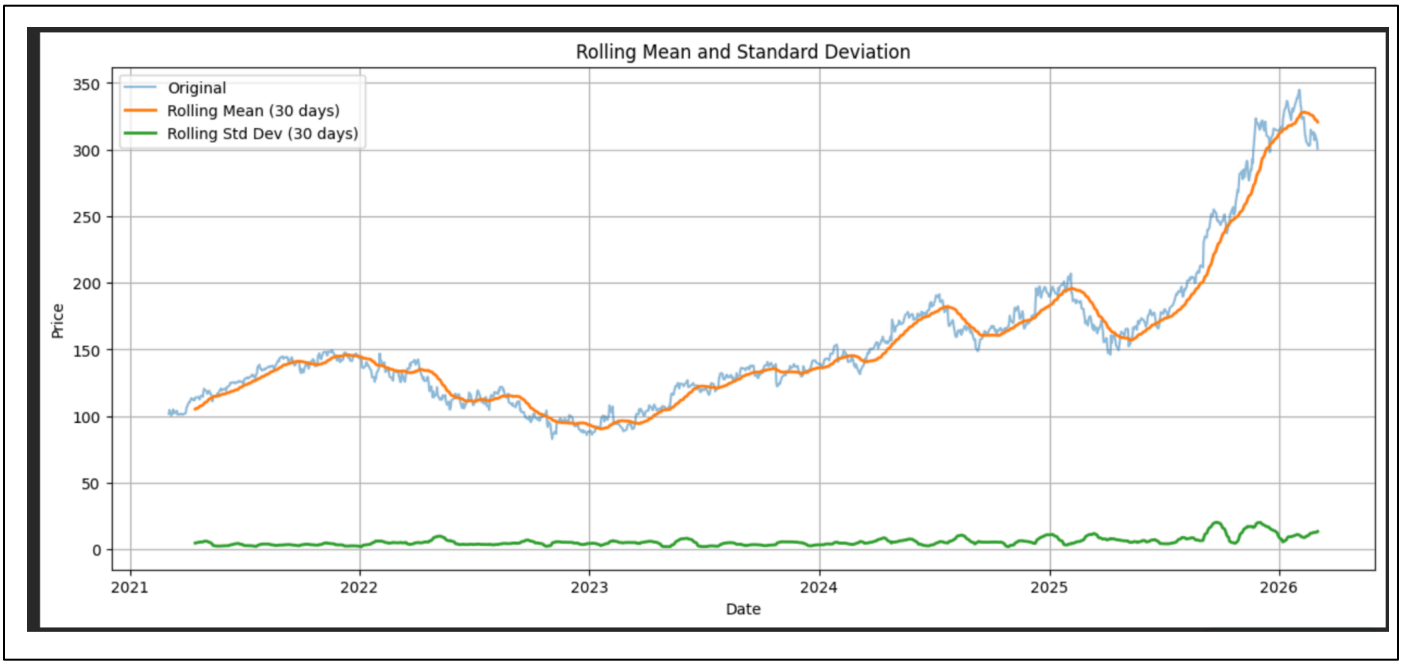
1. Does the average closing price remain constant over time, or does it drift?
2. Is the variance stable over time, or does the volatility cluster?
3. Are there distinct high-volatility and low-volatility periods?

The following plot taken over the window of 30 days, provides insight into the time-varying behavior of the stock price series.

The upward movement of the **rolling mean** confirms that the **average closing price is not constant over time**, indicating a clear long-term trend and **non-stationarity**.

Rolling standard deviation increases during certain periods. These fluctuations in the rolling standard deviation reveal that volatility changes across different periods, with noticeable spikes during times of rapid price movement. This pattern demonstrates the presence of **heteroskedasticity and volatility clustering**. This **supports hypothesis H2 (non-constant variance)**.

The Rolling SD plot shows that there are periods of calm regimes (low-volatility periods) and turbulent regimes(high volatility periods). The market does not behave uniformly across time. Thus VaR models or models that can adapt to changing variance might perform better.



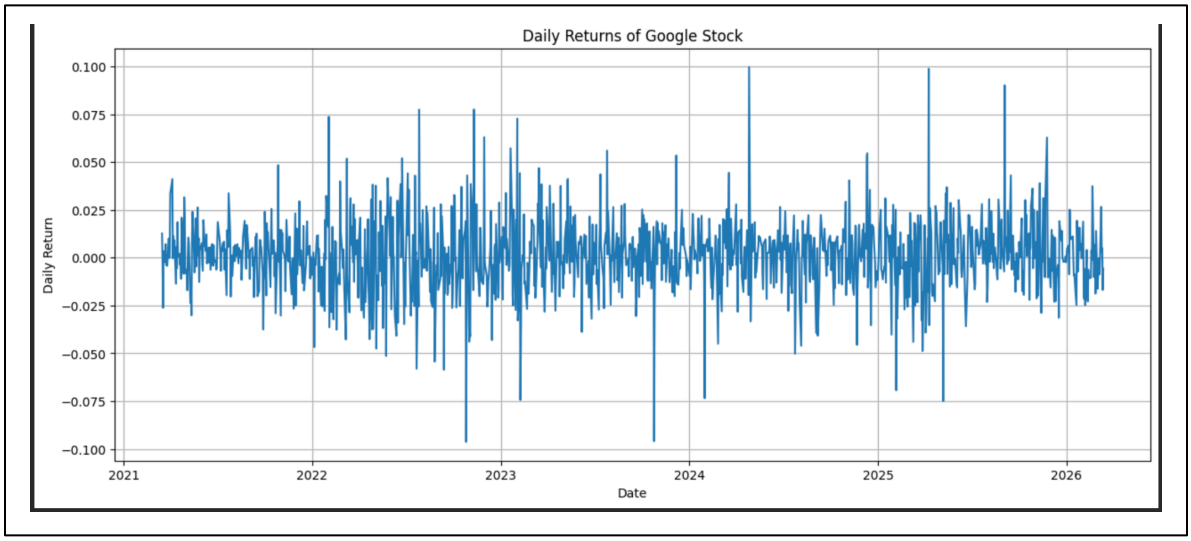
3.Daily Returns Analysis:

With the following two plots we are going to answer a bunch of questions:

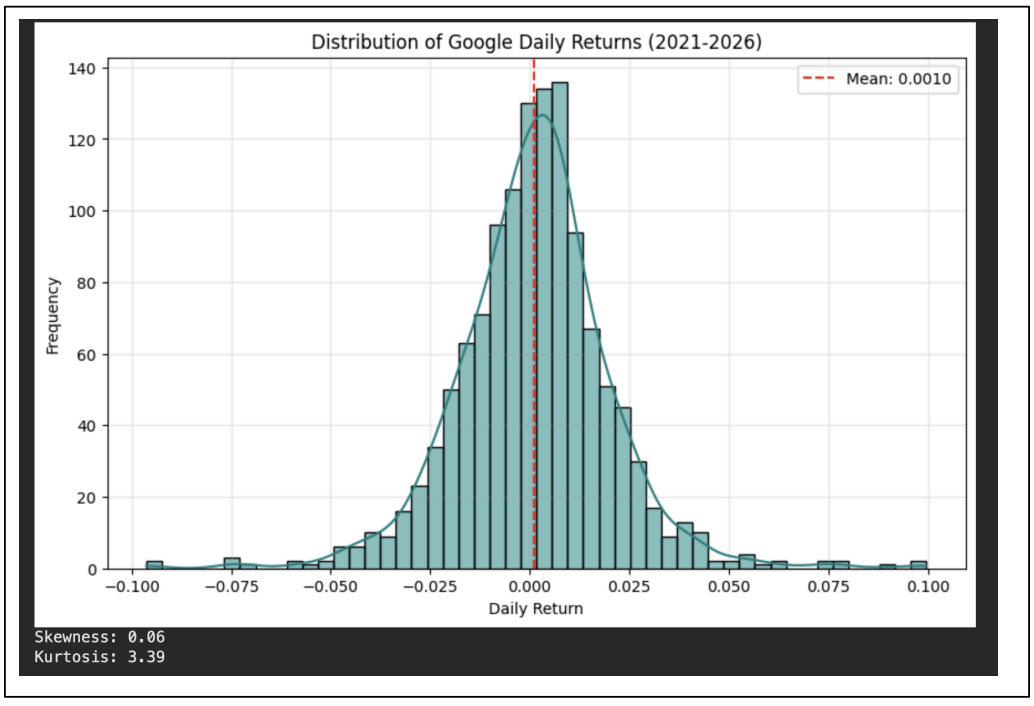
1. Do Daily Returns fluctuate around zero?
2. Is the Distribution of Daily Returns normally distributed?
3. Does volatility change over time?
4. Does the stock experience market shocks? I.e the tail behavior.

The daily returns plot shows that **Google's stock returns fluctuate around zero without a clear trend**, indicating that the **returns are approximately stationary**. This contrasts with the non-stationary behavior observed in the closing price series. Additionally, the plot reveals periods of increased volatility followed by calmer periods, a phenomenon known as volatility clustering. These characteristics are consistent with classical financial time series behavior. The apparent randomness of returns supports the use of the

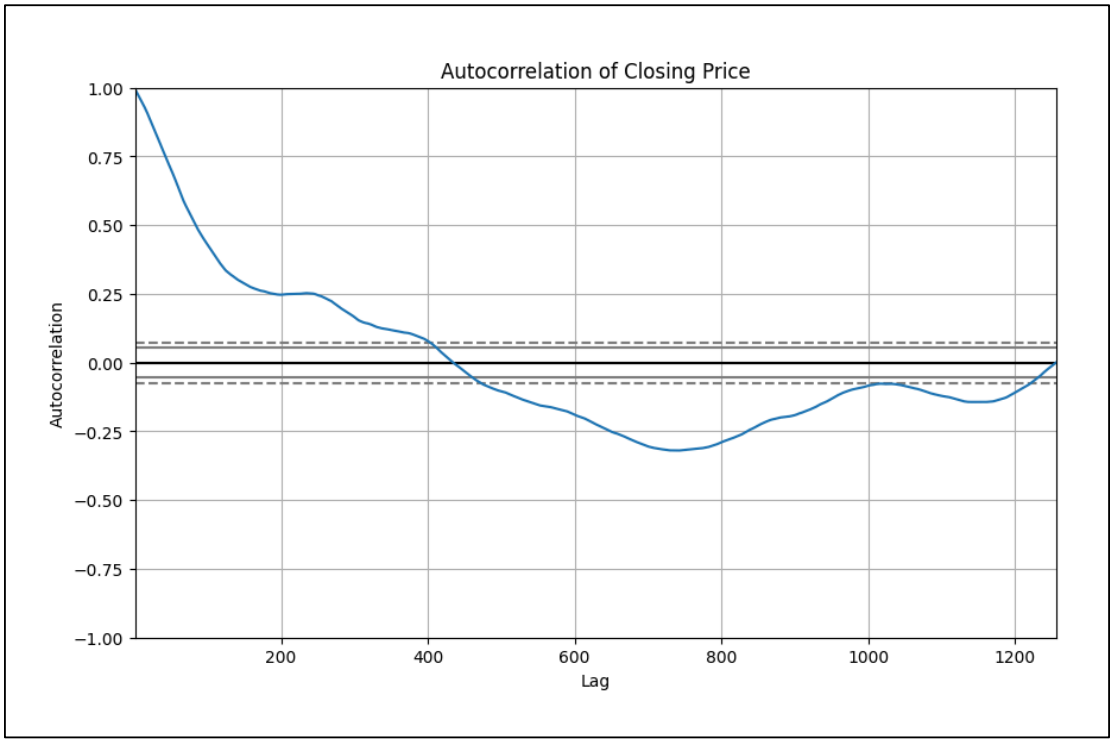
Random Walk model as a strong naïve baseline, while the presence of temporal patterns in volatility justifies the use of more advanced models such as ARIMA, XGBoost, and LSTM for forecasting (supporting H5).



The distribution of **Google's daily returns shows that returns are centered very close to zero, with a mean of about 0.0010**, indicating that average daily price changes are small, supporting **H2**. The skewness of 0.06 suggests the distribution is nearly symmetric, with only a very slight right skew, meaning positive and negative returns are fairly balanced overall. At the same time, **the kurtosis of 3.39 indicates a leptokurtic** distribution, meaning the returns have heavier tails than a normal distribution and therefore experience extreme gains or losses more often than Gaussian assumptions would predict. Overall, this plot suggests that **Google's daily returns are approximately centered and symmetric but still exhibit meaningful tail risk**, which is a typical stylized fact of financial time series.



4.Autocorrelation Analysis: Does the past influence the future prices?



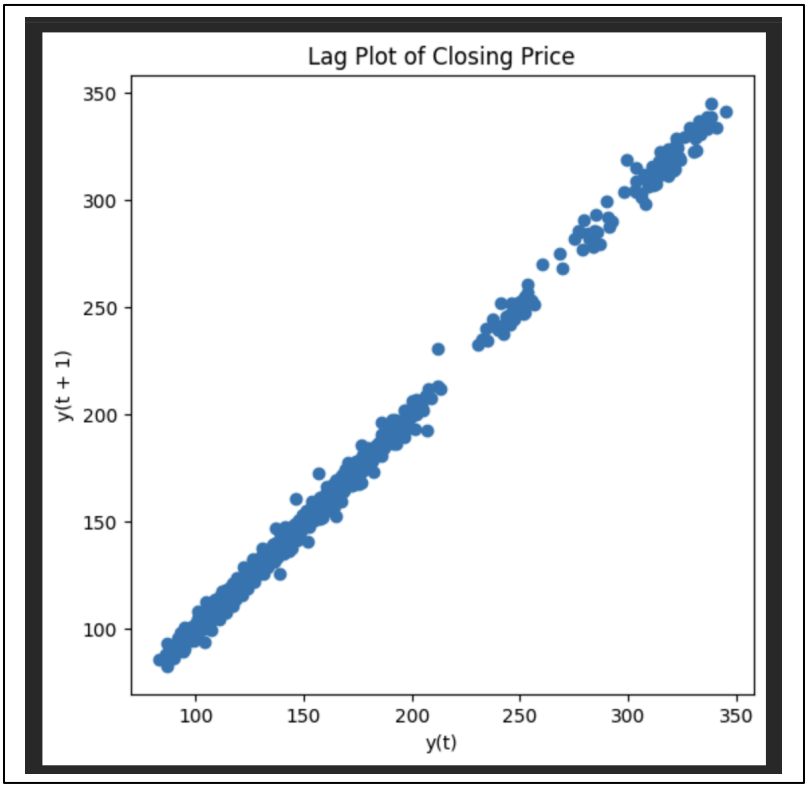
This **autocorrelation plot is answering whether Google's closing price is serially dependent over time**, meaning whether past prices remain informative about future prices across different lags. The very high positive autocorrelation at small lags shows strong persistence in the price series, which is expected because stock prices evolve smoothly rather than jumping independently from day to day. The slow decay of autocorrelation, rather than an immediate drop to zero, indicates that the **raw closing price series is non-stationary** and still contains strong trend information. As the autocorrelation eventually crosses zero and becomes negative at larger lags, it suggests long-horizon reversal relative to earlier periods rather than true cyclical predictability. Overall, this plot shows that the raw price series is highly dependent and trending, which justifies transforming the data through differencing or log returns before applying forecasting models such as ARIMA or LSTM.

5.Lag-plot analysis of the closing price:

The lag plot answers whether the current closing price y(t) has a strong relationship with the previous day's closing price y(t+1), or equivalently **whether the series exhibits serial dependence from one time step to the next.** In other words, it asks whether consecutive observations move independently or whether today's price is highly predictable from yesterday's price. This is an important diagnostic for checking autocorrelation and understanding whether the raw price series contains persistence, which directly affects model choice and the need for transformations such as differencing or log returns.

In the following plot the points lie almost perfectly along a straight upward-sloping line, indicating extremely strong positive lag-1 dependence in the closing price series. This means adjacent stock prices are highly similar and the series is strongly persistent, which is typical for raw financial prices because prices evolve gradually over time rather than randomly resetting each day. Such a tight linear pattern suggests that the

price series is not stationary and still contains trend structure, so modeling raw prices directly can be misleading. **This supports transforming the series into returns before forecasting, since returns reduce this persistence and make the data more appropriate for statistical and machine learning models.**

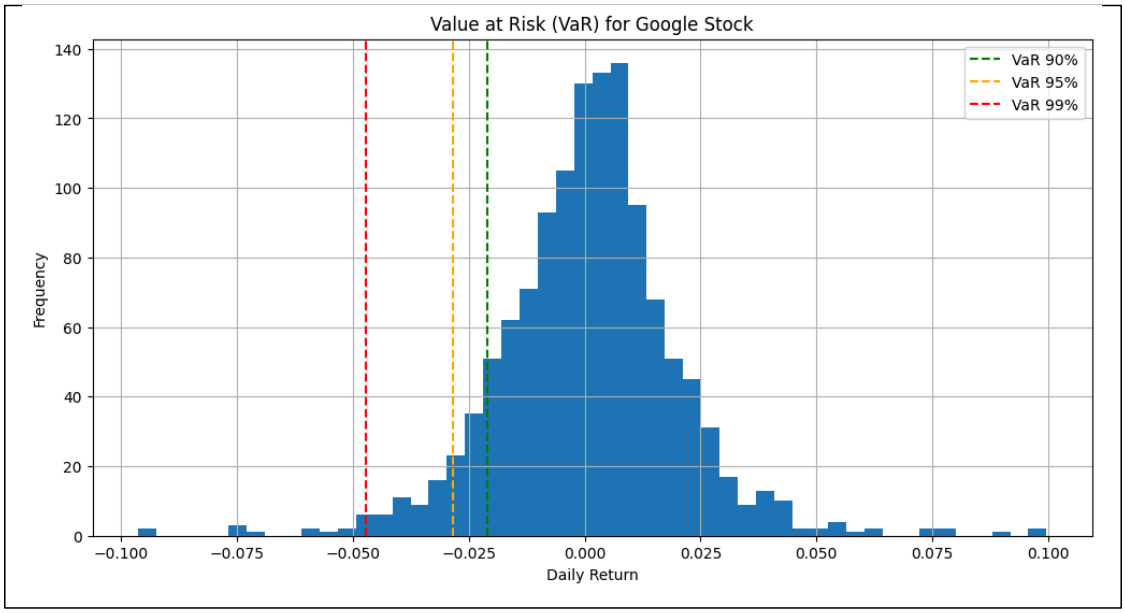


This lag plot mainly **supports H1:** Long-Term Behavior because, the very strong positive lag-1 relationship shows that consecutive closing prices are highly persistent and move smoothly over time, which is exactly what we expect in a trending price series rather than a purely random scatter. It shows that the stock price retains memory from one day to the next and does not fluctuate independently, reinforcing the idea of sustained directional movement over time**.**

6.VaR Analysis:

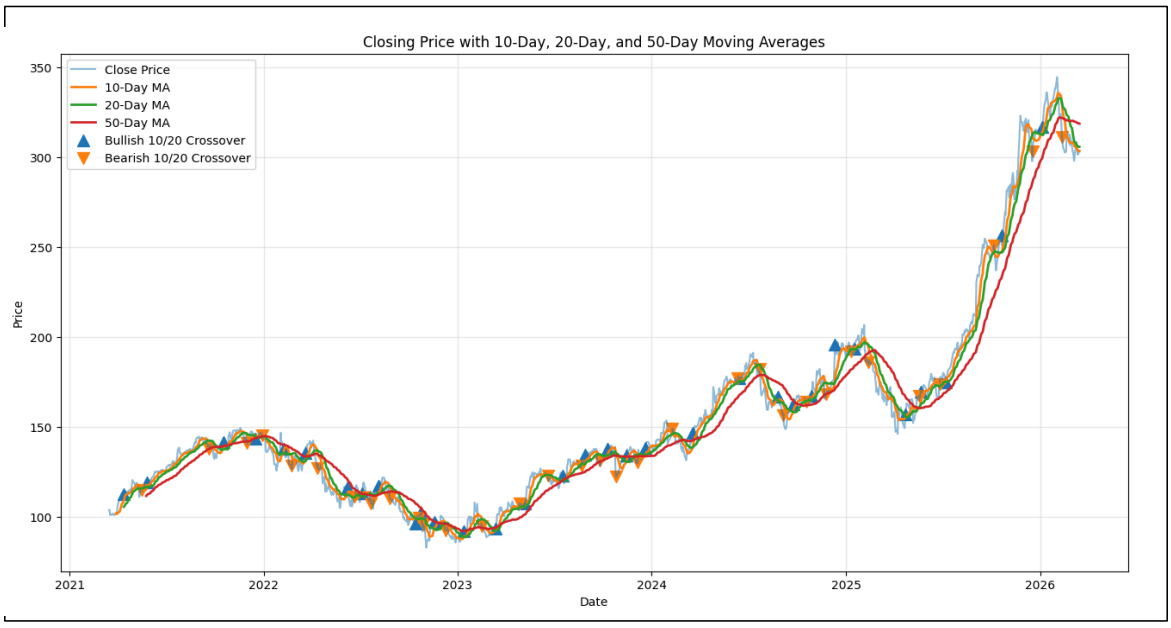
These VaR calculations and the accompanying histogram are designed to answer the question: **How much downside loss might an investor face in Google stock over a single day under normal market conditions at different confidence levels?** More specifically, they ask where the left-tail risk of the return distribution lies and how severe rare negative return events can be. By computing the empirical 90%, 95%, and 99% Value at Risk levels, the analysis quantifies threshold losses such that there is only a 10%, 5%, or 1% probability, respectively, of observing a worse daily return. This directly evaluates whether Google stock exhibits meaningful downside exposure rather than just average day-to-day fluctuation.
The results show that **Google's daily VaR is approximately -2.09% at 90% confidence, -2.84% at 95% confidence, and -4.72% at 99% confidence**, meaning that on most trading days losses should stay above these thresholds, but in rare cases losses can be substantially larger. On the histogram, these VaR cutoffs appear as progressively farther-left vertical lines, clearly marking increasingly extreme portions of the loss

tail. This is important because average return and volatility alone do not fully describe financial risk; VaR provides an interpretable measure of potential downside loss that is directly relevant for investors, portfolio management, and risk control. These findings **support H4: Downside Risk and Value at Risk (VaR)**, since **they confirm that the return distribution contains measurable tail risk consistent with volatility clustering and non-normal financial behavior**.



8. Moving Averages Analysis:

The following plot supports **H3 fairly well.**



The raw closing price series is visibly more jagged and noisy than the 10-day, 20-day, and especially the 50-day moving average curves. As the averaging window increases, the line becomes smoother: the **10-day**

MA follows short-term price movements closely, the **20-day MA** filters more of the daily fluctuations, and the **50-day MA** shows the broad underlying trend most clearly. That pattern is exactly what you would expect if moving averages are reducing short-term noise and highlighting directional movement. In your figure, this is especially clear during the decline from early 2022 into early 2023, the recovery through 2023–2024, and the strong upward trend into late 2025 and early 2026. The 50-day moving average lags more, but it makes the long-run bullish trajectory much easier to see.
The crossover markers also give visual evidence of **momentum shifts**. The bullish 10/20 crossovers tend to occur near the beginning of upward phases, while bearish 10/20 crossovers tend to appear around weakening or reversal periods. For example, around the 2022 downturn, the repeated bearish crossovers align with loss of upward momentum, while later bullish crossovers during 2023–2024 coincide with recovery and trend continuation. In the sharp rally into late 2025, the shorter moving average remains above the longer one for much of the advance, which is consistent with sustained positive momentum. Overall, the plot suggests that moving averages do smooth noise and that short-term versus long-term moving average crossovers can act as practical signals of changing trend direction, though they should be interpreted as supporting indicators rather than exact turning-point predictors.

B.Predictive Modeling Evaluation:

For this part of the project we first tried the models **to predict the closing price** and evaluated everything using the following common Evaluation Criterion.

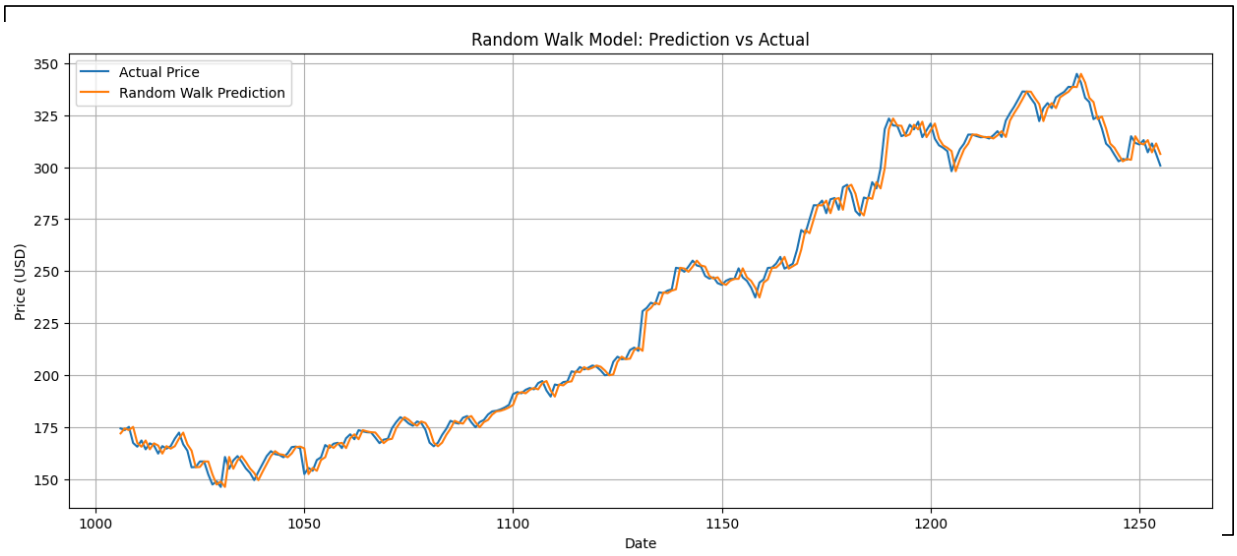
Evaluation Criterion:

We evaluate the forecasting models using **RMSE, MAE, and MAPE** because together they provide a balanced assessment of prediction error from complementary perspectives. **MAE (Mean Absolute Error)** measures the average absolute difference between predicted and actual prices, making it easy to interpret in the original units of the stock price and useful for understanding typical forecast error. **RMSE (Root Mean Squared Error)** also measures prediction error in the original units, but it penalizes large errors more heavily because the errors are squared before averaging, which is important in financial forecasting where occasional large misses can be especially costly. **MAPE (Mean Absolute Percentage Error)** expresses error as a percentage of the true price, allowing performance to be interpreted on a relative scale and making it easier to compare forecasting accuracy across models. These three metrics were chosen because they are standard, intuitive, and together capture average error magnitude, sensitivity to large deviations, and scale-independent percentage accuracy, giving a more complete evaluation than any single metric alone.

A. Random Walk Model:

The Random Walk model provides a strong naive baseline, as its predicted prices track the actual Google closing prices very closely over the test period, which is reflected in the relatively low error values of **RMSE = 4.31**, **MAE = 3.18**, and **MAPE = 0.0141**. This indicates that the model's forecasts differ from the true closing price by only about $3-4 on average, or roughly 1.4%, which is quite accurate given the scale of the stock price. The close overlap between the predicted and actual lines suggests that much of the short-term movement in stock prices can be approximated by assuming tomorrow's price will be very similar to today's price. This model is based on the assumptions that stock prices follow a martingale-like process, that the best forecast for the next period is the current observed price, and that future price changes are

unpredictable from past information beyond the most recent value. Because of these assumptions, the Random Walk serves as a strong benchmark, and any more advanced forecasting model must outperform it to show genuine predictive value.



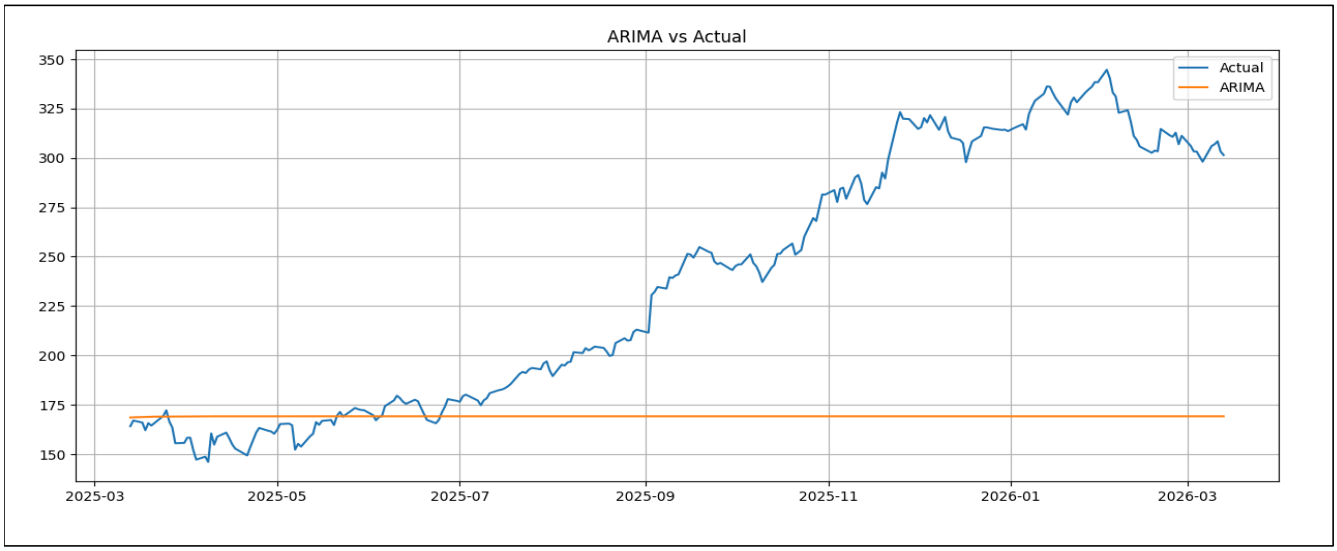
B. ARIMA Model:

We chose **ARIMA(1,1,1)** after the Random Walk model as a natural next-step benchmark that adds controlled statistical structure while still remaining simple and interpretable. The Random Walk assumes tomorrow's price is just today's price, with no extra learnable dynamics, whereas ARIMA(1,1,1) relaxes that assumption by first **differencing once** to handle the non-stationarity in stock prices and then modeling the differenced series using both an **autoregressive term** and a **moving-average term**. In other words, it allows tomorrow's change in price to depend not only on the most recent past change but also on the most recent forecast error. We chose (1,1,1)(1,1,1)(1,1,1) because stock prices typically require **one level of differencing** to remove trend, and using one AR and one MA term gives a parsimonious classical model that can capture short-range linear dependence without becoming unnecessarily complex. This makes it an appropriate model to test whether a standard econometric approach can improve on the naive Random Walk before moving to more flexible nonlinear models such as XGBoost and LSTM.

Before applying the model, The **Augmented Dickey-Fuller (ADF) test** was conducted to evaluate whether the closing price series is stationary. The test produced a **p-value of 0.9926**, which is significantly higher than the 0.05 significance level. Therefore, we fail to reject the null hypothesis that the series is non-stationary. This confirms that the **stock price series contains a unit root and exhibits non-stationary behavior.** As a result, differencing will be required before fitting ARIMA models.

Next we applied first order differencing on the Closing price. **After applying first-order differencing, the Augmented Dickey-Fuller test produced a p-value of 0.0,** which is well below the 0.05 significance level. This allows us to reject the null hypothesis of non-stationarity and conclude that the **differenced series is stationary.** This confirms that the original stock price series is integrated of order one, I(1), and that ARIMA models with differencing parameter d = 1 are appropriate.

Finally we fit the model and forecasted the closing price.
The ARIMA model performed very poorly for this forecasting task, with **RMSE = 93.15**, **MAE = 71.38**, and **MAPE = 0.2537**, indicating that its predictions deviate substantially from the true Google closing price in both absolute and relative terms. This poor performance is also clearly visible in the plot, where the ARIMA forecast remains almost flat around **$169** while the actual stock price rises sharply over time, eventually exceeding **$300**. Rather than tracking the upward movement and fluctuations in the series, the model effectively underfits the data and fails to adapt to the changing price level. This suggests that the linear structure assumed by ARIMA(1,1,1), even after differencing, is not sufficient to capture the complex, non-linear, and dynamic behavior of the stock price in this setting. Compared with the Random Walk baseline, ARIMA clearly underperforms, showing that adding a simple classical time-series structure did not improve forecasting accuracy for this dataset. **This highlights a limitation of classical linear time-series models when the underlying process exhibits nonlinearity or regime shifts**. These results motivate the use of more flexible approaches such as XGBoost with engineered lag features and LSTM sequence models.

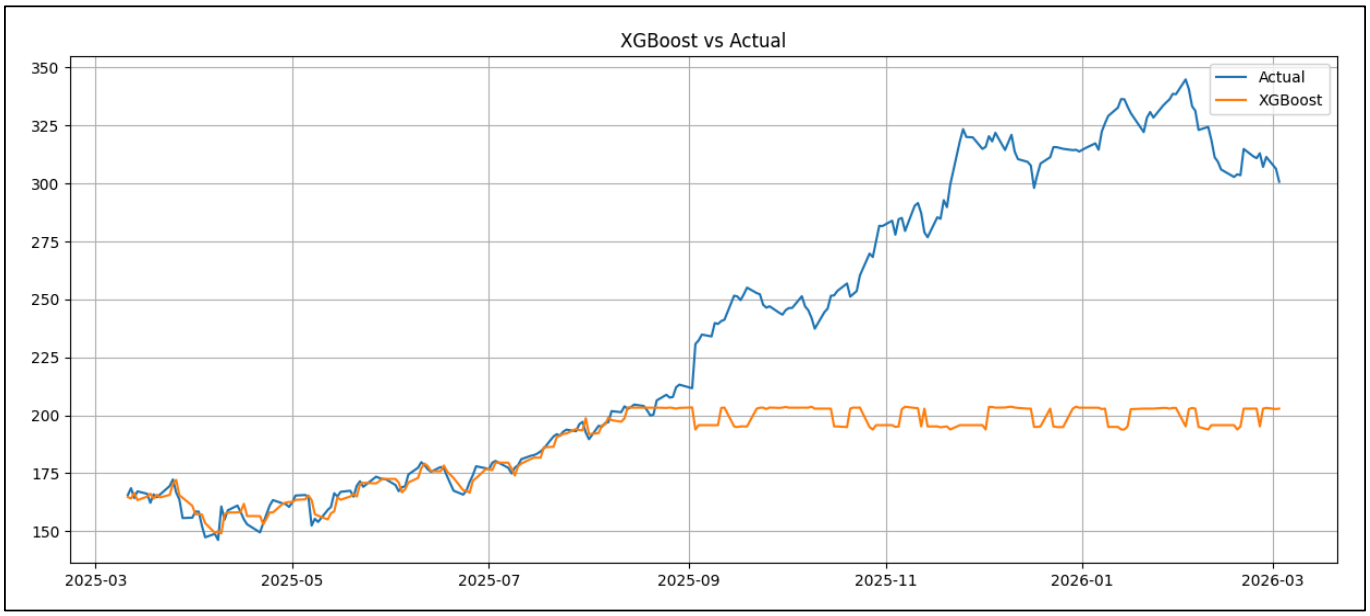


C. XGBoost Model:

For the XGBoost model, feature engineering was done by converting the univariate stock price series into a supervised learning dataset using time-based predictors derived only from past information. Specifically, **three lag features (`lag_1`, `lag_2`, `lag_3`) were created to capture short-term temporal dependence**, a 7-day and 30-day **moving average (`ma_7`, `ma_30`) were added to represent short and medium-term trend behavior**, and a 7-day **rolling standard deviation (`std_7`) was included to measure recent volatility**. Because these rolling and lagged features naturally create missing values at the beginning of the series, those rows were dropped, reducing the dataset from 1256 to 1227 observations. This feature engineering step is important because **XGBoost does not inherently understand time order the way sequence models do, so we explicitly provide historical structure, local trend, and volatility information as predictors.**

The XGBoost results still show weak forecasting performance, with **RMSE = 69.88**, **MAE = 48.09**, and **MAPE = 0.1633**, meaning the model's predictions are, on average, far from the true stock prices, especially in the later part of the test period. In the plot, the model tracks the early section reasonably well when prices remain within the historical range seen during training, but once the **actual series begins rising sharply after around September 2025,** the **predicted values flatten near t**he **195-205** range instead of following the upward movement toward **300+**. This indicates that the model learned local historical patterns from the lag, moving average, and volatility features, but it could not extrapolate to a new price regime outside what it had frequently observed during training.

This behavior is important because it highlights a common limitation of tree-based models in time-series forecasting: they are generally strong at learning non-linear relationships within the range of the training data, but they are poor at extrapolating sustained upward or downward trends beyond that range. So even though XGBoost performs better than the ARIMA model in this experiment, it still underperforms badly relative to the Random Walk baseline because it fails to adapt to the strong upward trend in the stock. In hypothesis terms, this provides only weak support for the idea that machine learning models can outperform traditional linear models, since XGBoost does beat ARIMA here, but it does **not** support the stronger hypothesis that advanced predictive models meaningfully outperform a simple baseline for short-term stock forecasting on this dataset.



D. LSTM model to predict the Closing Prices:

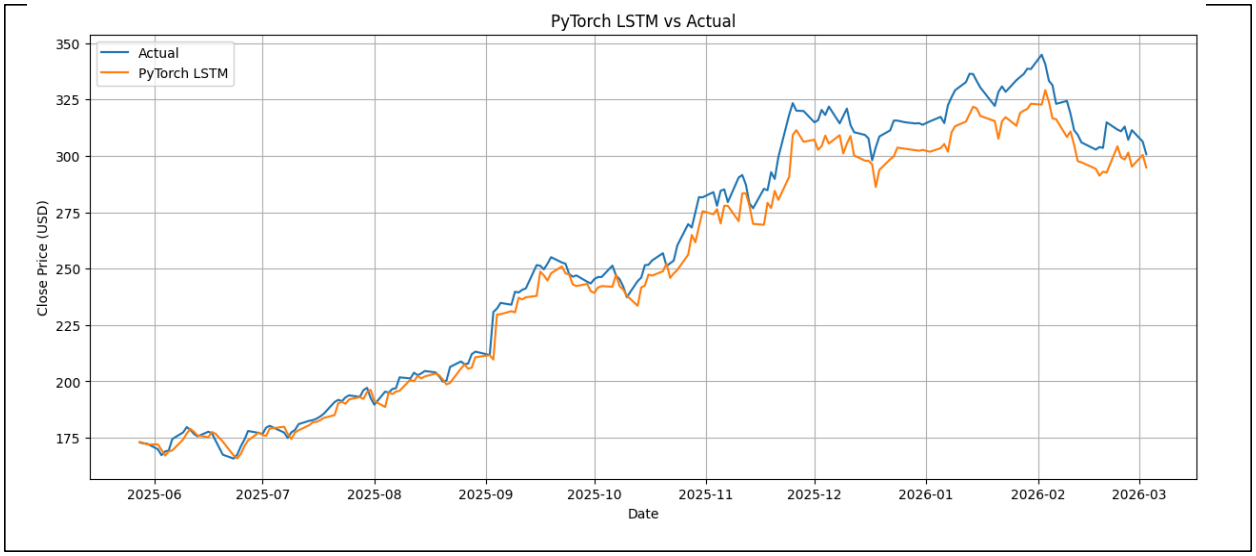
Moving on we chose an **LSTM (Long Short-Term Memory)** model because stock prices are sequential time-series data, where the current value depends not just on the immediately previous day but on patterns that evolve over many past days. Unlike Random Walk, ARIMA, or tree-based models such as XGBoost,

LSTM is specifically designed to learn temporal dependencies from ordered sequences and can, in principle, capture both short-term fluctuations and longer-term trends. This makes it a strong candidate for forecasting closing prices, especially when the data may contain non-linear relationships, momentum effects, and evolving patterns that simpler linear models cannot model well. The chosen architecture was a relatively simple **PyTorch based LSTM** with **input size = 1**, **hidden size = 64**, and **1 LSTM layer**, followed by a fully connected linear layer that maps the final hidden representation to the next predicted closing price. This architecture was selected as a reasonable baseline deep learning model: simple enough to train stably on a univariate dataset, but expressive enough to learn sequential dynamics over time.

To apply the model, the closing-price series was first scaled and then converted into a **windowed supervised dataset** using a **60-day lookback period**. In other words, for each prediction, the model was given the previous 60 closing prices as input and asked to predict the next day's closing price. This transformed the raw time series into input-output pairs of shape **(samples, 60, 1)** for the LSTM, where each sample is a 60-step sequence with one feature per timestep. After sequence generation, the training set contained **944 sequences** and the test set contained **192 sequences**. The LSTM was trained using **MSELoss** and the **Adam optimizer** with learning rate **0.001**, and during prediction the trained model was fed the test sequences to generate outputs in scaled form, which were then transformed back to the original price scale for evaluation. The resulting predictions were compared with actual closing prices using **RMSE, MAE, and MAPE**, and then plotted against the true values to visually assess tracking performance.

The results show that the LSTM performed substantially better than ARIMA and XGBoost, and reasonably close to the actual series, with **RMSE = 9.47**, **MAE = 7.25**, and **MAPE = 0.0256**, meaning the average **percentage prediction error is about 2.56%**. In the plot, the predicted line follows the actual price movement quite closely across most of the test period, capturing the general upward trend, several local turning points, and even much of the medium-term variation. **This suggests that the model successfully learned meaningful sequential structure from the previous 60 days of prices and was able to convert that information into fairly accurate next-step forecasts.** Compared with the flat ARIMA prediction and the under-responsive XGBoost forecasts, the LSTM clearly handles temporal dynamics much more effectively.

At the same time, the plot also shows an important limitation: the LSTM predictions are smoother and slightly lower than the actual values during periods of sharp upward jumps or high volatility, especially in late 2025 and early 2026. This means the model captures direction and trend well, but tends to underpredict extreme peaks and react more conservatively to abrupt market movements. That is typical for many sequence models trained with MSE loss, since they often learn the dominant pattern and smooth out sudden deviations. **Overall, the LSTM provides strong evidence that deep learning can model stock-price sequences better than traditional linear or manually engineered-feature approaches in this dataset, and it supports the hypothesis that models capable of learning non-linear temporal dependencies are more effective for stock forecasting than simpler baselines.**



Directional Accuracy:

Directional accuracy measures **how often a model correctly predicts the direction of movement** of the **stock price from one day to the next, regardless of whether the predicted price is numerically very close to the true price**. In other words, it checks whether the model correctly says the price will go **up or down**, not how large that change will be. This is important in financial forecasting because, for many trading or decision-making settings, getting the direction right can matter as much as or more than minimizing absolute error. From the results, the LSTM has the highest directional accuracy (**0.5288**, or about **52.9%**), followed closely by the Random Walk model (**0.5221**, or about **52.2%**), while XGBoost (**40.4%**) and ARIMA (**35.6%**) perform much worse. This shows that LSTM is the best model overall in terms of both price-error metrics and directional prediction, but its directional accuracy is still only slightly better than random guessing, which would be around **50%** in a roughly balanced up/down setting.

The directional accuracy results provide a **strong justification for moving from a closing price-based LSTM to a Return-LSTM** because they show that, in stock forecasting, correctly predicting the **direction** of movement can be as important as minimizing raw price error. Even though the price-based LSTM achieved the best overall RMSE, MAE, and MAPE, its directional accuracy was only modestly above 50%, suggesting that modeling prices directly may smooth the series and capture level trends without fully learning the day-to-day up/down behavior that matters in financial prediction. Since returns are a more stationary and movement-focused representation of the series, modeling **log returns** instead of raw prices is a natural next step: it shifts the learning task from reproducing price levels to capturing the underlying percentage changes and directional dynamics of the market. Therefore, introducing a Return-LSTM is justified as a way to better align the modeling objective with directional forecasting performance and potentially improve the model's ability to detect short-term market movements.

E. Return LSTM model:

A **return** measures how much a stock changes from one period to the next, relative to its previous value, so it captures movement rather than price level. In this case, the variable `logret` is the **log return**, computed as the difference between consecutive logarithms of closing prices, which is approximately the percentage change when daily movements are small. We use **log returns** instead of raw prices because **returns are usually more stable and closer to stationary, which makes them more suitable for time-series modeling**. Log returns are also additive over time, meaning multi-period returns can be summed, which is mathematically convenient for forecasting and financial analysis. By modeling returns rather than prices, we focus directly on the stock's daily gain or loss behavior, which is more aligned with volatility, risk, and directional prediction.

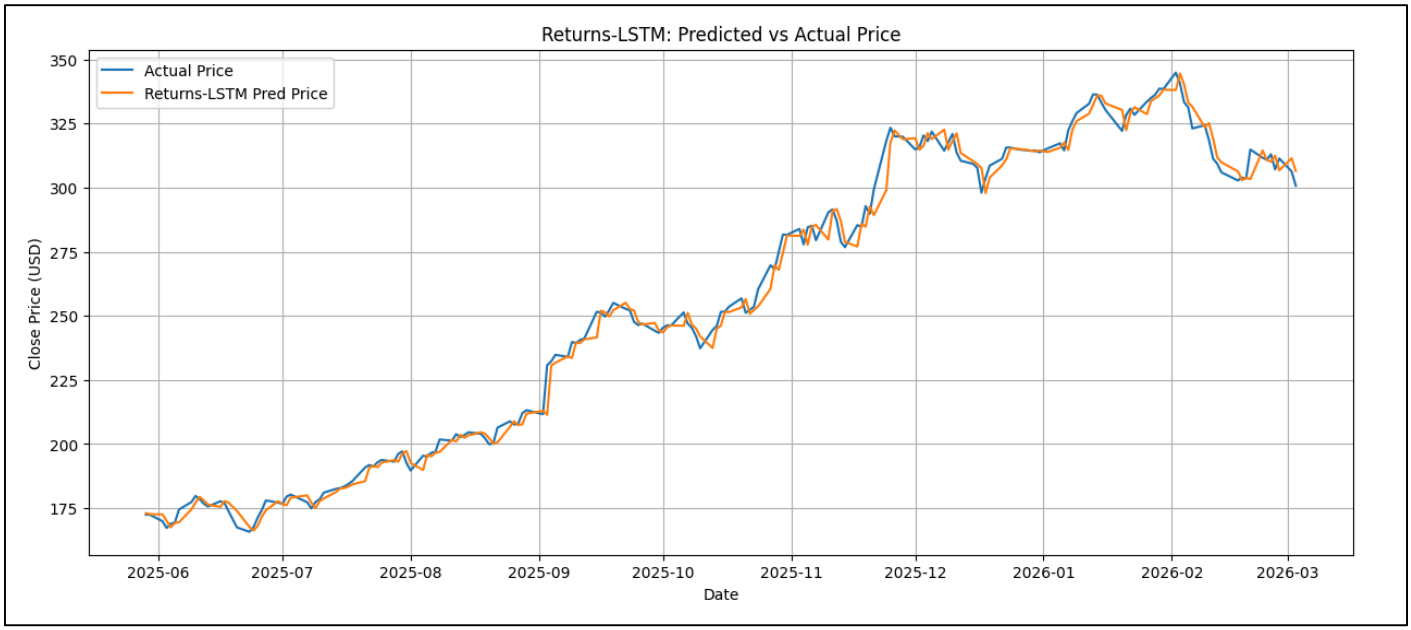
The summary statistics of the variable `logret` suggest that the **return distribution is centered very close to zero**, with a **mean log return of about 0.000873,** indicating a **small positive average daily growth**, while the **standard deviation of about 0.0193** shows that day-to-day fluctuations are much larger than the mean. The **median return is also close to zero (0.001618)**, and the **middle 50% of daily returns** lie roughly between **-0.96%** and **1.12%**, so most daily movements are relatively small. However, the **minimum** (**-10.13%**) and **maximum** (**9.50%**) values show the presence of occasional large shocks, meaning the distribution has heavier tails than a simple normal distribution would suggest. Overall, the return series appears much more tightly centered and stable than the raw price series, but it still contains volatility and extreme movements, which is exactly why using returns is useful for models like Return-LSTM that aim to learn short-term market dynamics rather than long-run price level drift.

The **Return-LSTM** uses essentially the same neural architecture as the earlier price-based LSTM, but the target variable is changed from raw closing price to **daily log return**. This was done because returns are more stationary, more centered around zero, and more directly tied to market movement than raw prices, which makes them a better signal for sequential learning. The model architecture remained a simple PyTorch LSTM regressor with **input size = 1**, **hidden size = 64**, and **one LSTM layer**, followed by a linear output layer that maps the final hidden state to a single predicted next-day return. This design is appropriate because the LSTM can learn temporal dependencies across a sequence of past returns, while the small architecture keeps the model interpretable and stable enough for a univariate forecasting task.

To apply the model, the closing-price series was first transformed into **log returns** using `log(Close).diff()`**,** and the missing first observation was dropped. The return series was then split into train and test sets, scaled, and converted into **60-day rolling sequences**, exactly like before, except now each input window contained the previous 60 returns rather than 60 prices. After training the LSTM on these return sequences, the model generated predicted returns on the test set in scaled form, which were inverse-transformed back to the original log-return scale. Since the evaluation metrics were still to be reported in terms of **closing price prediction**, the **predicted returns were converted back into predicted closing prices** using the compounding relationship $P_{t+1} = P_t \, exp(r_{t+1})$, where the previous actual closing price served as the base price. This allowed the Return-LSTM to be evaluated directly against the actual observed stock prices using RMSE, MAE, MAPE, and directional accuracy.

The results show that the Return-LSTM performed extremely well and was the strongest model overall in terms of price forecasting error after reconstruction. Its metrics were **RMSE = 4.389**, **MAE = 3.176**, and **MAPE = 0.01228**, which are all slightly better than the Random Walk baseline and much better than ARIMA, XGBoost, and the earlier price-based LSTM. The plot confirms this: the predicted price curve almost overlaps with the actual closing-price series throughout the test window, following both the upward trend and the short-term fluctuations very closely. This indicates that modeling returns first and then reconstructing prices helped the model capture the local dynamics of the stock more effectively than directly modeling raw price levels.

At the same time, the **directional accuracy** of the Return-LSTM is about **0.5053**, which is only slightly above random chance and lower than the price-based LSTM and Random Walk. This means that while the **Return-LSTM is excellent at producing numerically accurate price paths, it does not substantially improve the ability to predict whether the market will move up or down on the next day. In other words, the model captures magnitude and continuity of the series very well after price reconstruction, but it still struggles with pure directional classification. So the Return-LSTM gives the best overall regression performance and supports the idea that return-based modeling is more appropriate for price forecasting,** but it also suggests that improving directional prediction may require additional features, classification-specific objectives, or hybrid models beyond a univariate return sequence alone.



Conclusion:

In conclusion, our study finds that model choice and data representation have a major impact on stock forecasting performance. Among all methods tested, the **Return-LSTM** achieved the best overall regression accuracy, with the **lowest RMSE, MAE, and MAPE** after converting predicted returns back into prices, while the **price-based LSTM** also performed strongly and clearly outperformed ARIMA and XGBoost. The **Random Walk** baseline remained highly competitive, which reinforces an important financial insight:

short-term stock prediction is difficult, and simple persistence models are often hard to beat. In evaluating our hypotheses, the results provide **strong support** for the hypothesis that deep learning models can capture temporal structure more effectively than traditional linear models such as ARIMA, and **partial support** for the hypothesis that non-linear machine learning methods outperform classical time-series methods, since XGBoost did outperform ARIMA but still failed to match the strongest baselines. The findings also support the hypothesis that **return-based modeling is more suitable than price-level modeling** for this dataset, because the Return-LSTM gave the best numerical forecasting results. However, the hypothesis that advanced models would also achieve strong **directional prediction** is **not strongly supported**, since directional accuracy remained only slightly above random even for the best models. This suggests that while models can reconstruct price paths well, predicting next-day market direction remains much harder.

These findings are broadly consistent with prior research in financial forecasting, which has often shown that raw prices are non-stationary, returns are more appropriate for modeling, and deep learning architectures such as LSTMs can better learn sequential dependencies than classical linear approaches. At the same time, our results also align with the large body of literature emphasizing the strong efficiency and noise present in financial markets, where even sophisticated models struggle to produce robust directional forecasts. Several limitations should be acknowledged. First, the study used a **univariate setup**, relying mainly on historical closing prices or returns, so important explanatory information such as volume, macroeconomic indicators, news sentiment, technical indicators, and broader market variables was not included. Second, the analysis was conducted on a **single stock and a limited test window**, which restricts the generalizability of the conclusions. Third, the models were evaluated primarily with standard regression metrics and directional accuracy, but not with trading-based performance measures such as cumulative return, Sharpe ratio, or drawdown, which are often more meaningful in finance. Future research should therefore extend this work using **multivariate features**, broader cross-stock evaluation, **walk-forward validation**, and potentially more advanced architectures such as GRUs, attention-based models, or transformer-based time-series networks. A promising direction would also be to build separate models for **direction classification** and **return magnitude estimation**, since our results suggest that accurate price reconstruction and accurate directional forecasting are not necessarily the same task.

Code base:

Github Link to the codebase

References:

Reliably download historical market data from with Python
Classical Financial & Statistical Models
Deep Learning & LSTM for Financial Forecasting
Stock Market Analysis ETL- Github repository